GraphAccel: An In-Storage Accelerator for Efficient Graph-Based Vector Similarity Search Using Page Packing and Speculative Search Optimization

Yoonyoung Kwon*
Sungkyunkwan University
Suwon, Korea
kyy0307@g.skku.edu

Yunjong Boo*
Sungkyunkwan University
Suwon, Korea
qndbswhd1@g.skku.edu

Hyungmin Cho
Sungkyunkwan University
Suwon, Korea
hyungmin.cho@skku.edu

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132788

***Abstract*—Graph-based search for approximate vector similarity is essential in AI applications, such as retrieval-augmented generation. To support large-scale searches, vector search graphs are often stored on storage devices like SSDs. In this paper, we introduce GraphAccel, an in-storage accelerator optimized for efficient graph-based vector similarity search. Our architecture incorporates an optimized page packing mechanism to reduce SSD page accesses per query, alongside a speculative search scheme that maximizes utilization of idle SSD chips and channels. Through these optimizations, GraphAccel achieves notable performance improvements over existing SSD-based graph search solutions, including DiskANN and DiskANN++.**

***Index Terms*—In-Storage Processing, SSD Parallelism, Vector Search.**

I. Introduction

The rapid growth of unstructured data, such as images [1], text [2], and audio [3], has created a need for processing methods that preserve semantic information. Semantic vectors address this challenge by transforming data into high-dimensional representations that capture the relationships between data points. A critical operation on large-scale vector databases is finding similar vectors for a given query. Exact similarity search often relies on brute-force methods, which can be computationally expensive. As a result, many applications adopt approximate nearest neighbor search (ANNS) [4], [5], which is widely used in fields such as recommendation systems [6], computer vision [7], and retrieval-augmented generation (RAG) [8].

Many ANNS algorithms utilize graph-based approaches, which organize vectors into a navigable graph structure for indexing [9], [10]. When a query is issued, the search algorithm traverses this graph in two primary stages: *node expansion* and *distance calculation*. During the node expansion stage, the algorithm selects the next node to expand based on its proximity to the query, retrieving its vector and neighboring node information. In the distance calculation stage, the distance between the expanded node's vector and the query vector is computed, and candidate nodes are updated accordingly. By iteratively repeating these stages, the algorithm identifies the nodes in the graph that are closest to the query vector.

As dataset sizes continue to grow, large-scale vector search poses unique challenges: they are both data-intensive and exhibit complex, unpredictable access patterns across varying queries that complicate efficient processing. These characteristics demand high-throughput and low-latency solutions, especially as ANNS graphs are frequently stored on storage devices to support scalability.

Storing a vector database on flash-based storage devices, such as Solid-State Drives (SSDs), presents significant challenges. The inherently slow access latency of flash storage hampers search performance, as it involves repeated, unpredictable access to flash pages. To address this, effective strategies are needed to minimize the number of page accesses during a search. Additionally, the unpredictable read latency of SSDs necessitates dynamic mechanisms to maximize SSD utilization and reduce overall search latency.

In this paper, we introduce *GraphAccel*, an in-storage accelerator designed to address these challenges by optimizing graph-based vector similarity searches on SSDs. First, we propose a new packing mechanism that groups nodes that are accessed together frequently into the same flash page, reducing flash access roundtrips during query processing. Additionally, GraphAccel incorporates a search algorithm that leverages In-Storage Processing (ISP) capabilities by monitoring hardware resource status to maximize SSD parallelism, sending additional speculative requests to idle channels and chips. This approach minimizes total page accesses during queries, maximizes hardware resource utilization, and ultimately achieves high accuracy with low latency.

We evaluated GraphAccel's search performance using large-scale vector datasets, including Turing100M [11], SIFT1B [12], and DEEP1B [13]. Compared to its foundational systems, DiskANN [9] and DiskANN++ [14], GraphAccel achieves up to 80.5% and 73.4% reductions in search latency, respectively, while maintaining equivalent levels of recall quality.

II. Background

A. SSD Preliminary

SSDs store data using individual cells that operate through electrically controlled mechanisms for data operations. These cells adjust their stored charge to establish specific voltage levels, which are sensed to decode the stored data. While this electrical design enables efficient data handling, it also introduces reliability challenges such as *read disturbance* and *retention loss* [15], [16].

Read disturbance occurs when repeated read operations on a cell cause slight voltage shifts in adjacent cells, potentially altering their threshold voltage levels, risking data corruption. In contrast, retention loss refers to the gradual leakage of electrons from cells over time, which lowers the stored voltage levels and degrade data accuracy.

To mitigate these issues and maintain reliability, modern SSDs employ Error Correction Codes (ECC) and *read-retry* mechanisms [17]–[19]. When the number of error bits exceeds the ECC's correction capability, the read-retry mechanism is triggered. This mechanism works by adjusting the voltage thresholds during read operations to reduce misread cells, thereby decreasing the error bit count to within ECC's corrective capacity. Once reduced, ECC corrects the remaining errors, restoring data integrity.

* Both authors contributed equally to this work.

While read-retry is critical for ensuring reliability, it introduces trade-offs. Each retry involves re-performing the read operation, increasing overall read latency. The number of retries required depends on factors such as the severity of read disturbance and the extent of retention loss. Consequently, read-retry operations add variability and unpredictability to SSD read latency.

B. DiskANN Search

TABLE I: Notations used in this work.

Symbol	Description
B	Beamwidth
$\mathcal{C}$	Maxheap for search list
$\mathcal{R}$	Maxheap for Top-k results
v	Node in the search graph
$N(v)$	Set of v's neighbor nodes
$\vec{v_{pq}}$	PQ vector of v
$\vec{v_{full}}$	Full vector of v

DiskANN's graph search algorithm identifies the k-closest nodes to a query by iterative *node expansions* [9]. It maintains a search list $\mathcal{C}$ of size L, from which the closest node to the query is selected for expansion. Node expansion involves reading the neighbors of the selected node, calculating their distances to the query, and adding them to $\mathcal{C}$ as potential candidates. This process repeats until all nodes in $\mathcal{C}$ are expanded, and the algorithm reports the top k results in $\mathcal{C}$.

DiskANN further optimizes performance by utilizing two distance calculation methods. When a node is initially added to the search list as a neighbor of an expanded node, the algorithm calculates an approximate distance using product quantization (PQ) [20], a technique that reduces vector dimensionality and computational overhead. Nodes selected for expansion are subsequently re-evaluated using their full vectors for greater accuracy, while their neighbors continue to be assessed with PQ vectors. Fully expanded nodes evaluated with their full vectors are stored in a separate top-k result heap, $\mathcal{R}$.

Algorithm 1: DiskANN Vector Graph Search

1 **Input:** Query vector q, k of top-k, Search List size L, Graph G
2 **Output:** k nearest neighbors of a query q
3 $v_c \leftarrow$ central vertex of G;
4 maxHeap $\mathcal{C}$ max capacity with L;
5 maxHeap $\mathcal{R}$ max capacity with k;
`// Insert` v_c `with computed PQ distance`
6 $\mathcal{C}.insert(v_c, PQ\_dist(\vec{v_{c_{pq}}}))$;
7 **while** $\mathcal{C} - \mathcal{R} \neq \emptyset$ **do**
8 $v_s \leftarrow$ extract closest unexpanded node from $\mathcal{C}$;
`// SSD I/O request`
9 read page $\mathcal{P}$ containing $\vec{v_{s_{full}}}$, $N(v_s)$ from SSD;
`// insert` v_s `with computed full distance`
10 $\mathcal{R}.insert(v_s, full\_dist(\vec{v_{s_{full}}}))$;
`// insert` $N(v_s)$ `with computed PQ distance`
11 **for** v ***in*** $N(v_s)$ **do**
12 read page $\mathcal{P}'$ containing $\vec{v_{pq}}$ from SSD;
13 $\mathcal{C}.insert(v, PQ\_dist(\vec{v_{pq}}))$;
14 **end**
15 **end**
16 **return** $\mathcal{R}$

In the original DiskANN, PQ vectors are kept in memory, with only full vectors stored on disk. While feasible for smaller graphs due to PQ compression ($\frac{1}{4}\times$ in DiskANN), larger graphs may exceed memory limits. To support extremely large graphs as well, we assume *even PQ vectors reside on storage*.

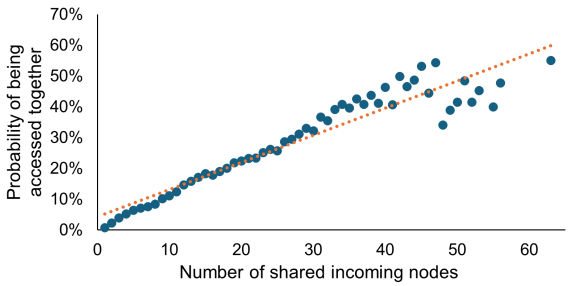


Fig. 1: Probability for a pair of nodes being accessed together based on the number of sharing incoming nodes.

Algorithm 1 and Table I summarize DiskANN's operation. The algorithm initializes with the graph's central node (line 3) and uses two max-heaps: $\mathcal{C}$ (capacity L) for PQ-evaluated candidates and $\mathcal{R}$ (capacity k) for top-k full-vector results (lines 4–5). The central node is inserted into $\mathcal{C}$ with its PQ distance (line 6). Node expansion is repeatedly applied (lines 8–14). The search proceeds while $\mathcal{C}$ contains candidates not yet in $\mathcal{R}$ (line 7).

III. Packing Algorithm of GraphAccel

The page size, the minimum granularity of read operations in flash storage, is typically larger than the size of a vector node. To optimize query performance, it is crucial to implement an efficient packing mechanism that groups vector nodes likely to be accessed together on the same page. Without such a mechanism, neighboring nodes spread across different pages would require additional I/O requests, leading to increased SSD roundtrips and degraded search performance.

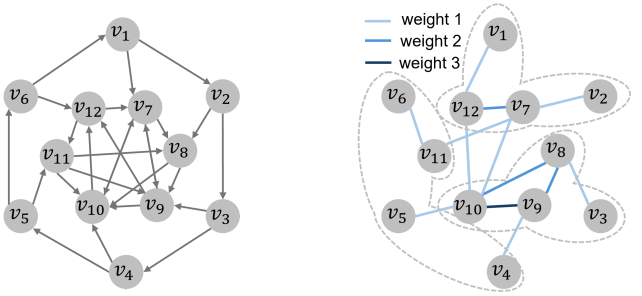
A. Starpacking

DiskANN++ addresses the packing issue by introducing an algorithm called *Starpacking*. Starpacking aims to co-locate each node with its $(b-1)$ nearest neighbors on the same page, where b represents the page capacity in terms of the number of vectors. The packing algorithm operates iteratively: starting with an unexpanded node, it groups the node with its $(b-1)$ closest unexpanded neighbors onto a single page. Once nodes are assigned to a page, they are marked as expanded, and the process continues with the remaining unexpanded nodes until all nodes are packed.

However, as Starpacking follows a greedy approach, the resulting packing is not globally optimal. The quality of the packing depends heavily on the order in which nodes are expanded. For nodes processed later in the sequence, where most neighboring nodes have already been marked as expanded, only a few of their neighbors can be placed on the same page. Consequently, query vectors that examine these nodes may require accessing a larger number of pages, resulting in increased I/O overhead and reduced search efficiency.

B. GraphAccel-Packing

The motivation for our GraphAccel-Packing algorithm stems from the insight that nodes frequently accessed together are not necessarily direct neighbors but rather nodes that share the same predecessor (incoming) node. Even if two nodes are not direct neighbors, if they share a common predecessor, they effectively become neighboring nodes relative to that predecessor. When the predecessor node is expanded, both of these nodes are accessed simultaneously.



(a) (b)

Fig. 2: GraphAccel-Packing example. (a) Directed graph with DiskANN indexing. (b) Transformed undirected graph with weighted edges.

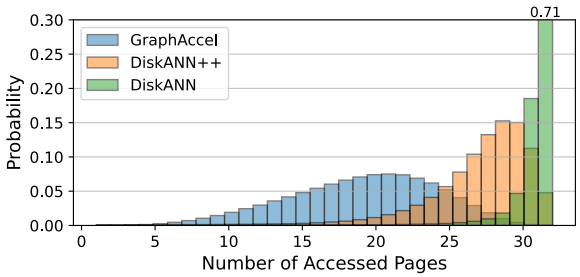


Fig. 3: Number of accessed pages when expanding a node with 32 neighbors.

To support this intuition, Fig. 1 illustrates the probability of two nodes being accessed together during query processing as a function of the number of shared incoming nodes. The probability is evaluated during search operations on the SIFT1M dataset using DiskANN indexing. The results demonstrate a strong positive correlation: the greater the number of shared incoming nodes, the higher the likelihood of the two nodes being accessed together.

Building on this motivation, we propose a packing mechanism that reframes the vector packing problem as a graph partitioning problem. Specifically, we transform the original directed graph into an undirected graph, where the edge weights are determined by the number of shared incoming nodes between connected nodes. We then partition this transformed undirected graph into sub-graphs each containing up to b nodes, and assign each sub-graph to a page.

Figure 2 illustrates the process of our packing method. The starting point is a directed graph constructed using DiskANN indexing, as shown in Fig. 2a. This graph is then converted into an undirected graph by connecting nodes that share common incoming nodes. The edge weights in the converted graph represent the number of shared incoming nodes between two connected nodes.

For example, while there is no direct edge between v_2 and v_7 in the original graph, both are neighbors of v_1 and will be accessed together when expanding v_1. Thus, in the converted graph, we introduce an edge between v_2 and v_7, enabling them to be packed on the same page. Similarly, for nodes sharing multiple incoming nodes, higher edge weights are assigned to reflect their higher likelihood of being accessed together. For instance, the edge weight between v_9 and v_{10} is three, as they share v_7, v_8, and v_{11} as incoming nodes.

Once the graph is transformed, we utilize PulP [21], a tool designed for large-scale graph partitioning, to partition the graph while minimizing the cut weights.

Before presenting a full evaluation of our packing mechanism in Sec. V, we provide a preview of its advantages over Starpacking in Fig. 3. The figure compares the number of pages accessed when expanding a node's neighboring nodes, using the SIFT1M dataset where each node has up to 32 neighbors. In DiskANN, which employs no specialized packing mechanism, neighboring nodes are distributed arbitrarily across pages. This results in up to 32 pages being accessed per node, causing a significant number of SSD I/O requests for each access.

While DiskANN++ reduces the number of accessed pages using Starpacking, nodes still require an average of 27.23 pages. This limitation arises because Starpacking maintains strong connectivity for nodes processed earlier in the packing process, but connectivity becomes fragmented for nodes processed later.

In contrast, our GraphAccel-Packing method significantly minimizes the number of pages required to access a node's neighbors. On average, our approach reduces the number of accessed pages per query by approximately 58.4% compared to DiskANN++ and by 72.8% compared to DiskANN, demonstrating its superior efficiency in packing and query performance.

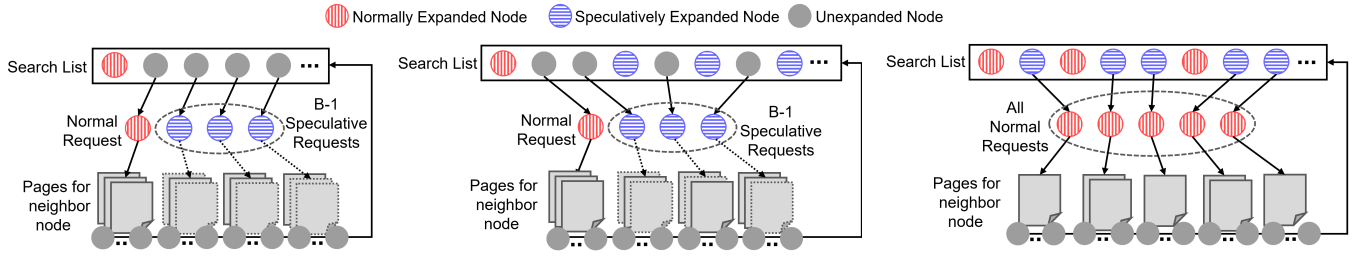
A downside of our approach is the requirement for large-scale graph partitioning. To enable partitioning on large-scale search graphs, we divide a large graph into multiple subgraphs, disregarding inter-subgraph connections. While this approach may not fully retain the original graph's structure, it only impacts the packing quality, leaving search accuracy unaffected. When splitting the graph into subgraphs, fewer but larger subgraphs preserve more of the original graph structure. However, larger subgraphs also increase computation time for calculating edge weights due to the greater number of nodes within each subgraph. To balance the trade-off between preserving graph information and managing computational overhead, we set the subgraph size to 50 million nodes. With this configuration, the calculation and storage of edge weights for each subgraph, as well as the subsequent packing process, required an average of 12 hours per subgraph. To expedite the process, all subgraph partitioning operations are executed in parallel.

IV. GraphAccel-Search

Disk-based ANN graph search is inherently constrained by the performance of disk access. To improve search performance, it is essential to exploit the abundant parallelism available in SSDs at both the channel and chip levels. However, the basic search mechanism, as outlined in Algorithm 1, fails to take full advantage of this parallelism due to its inherently sequential nature. Specifically, it computes and selects the next closest node for expansion only after completing disk accesses for the current closest node.

To address this limitation, DiskANN introduces the *Beamsearch* mechanism, which partially leverages SSD parallelism by expanding multiple nodes simultaneously. However, Beamsearch has inherent constraints, as it treats all node expansions equally, lacking the ability to prioritize critical accesses over non-critical ones.

In this work, we present *GraphAccel-Search*, a novel mechanism designed to fully utilize SSD parallelism through speculative search executions. By distinguishing between critical and speculative node expansions and dynamically managing SSD resources, GraphAccel-Search ensures that critical requests are prioritized while idle SSD resources are opportunistically used for speculative expansions. This approach overcomes the limitations of Beamsearch and delivers



(a) (b) (c)

Fig. 4: Phases of node expansion in GraphAceel-Search. (a) Initial phase after expanding the entry node. (b) Middle phase of the search. (c) Batch re-examination phase.

significant performance improvements for disk-based ANN graph search.

A. Limitations of Beamsearch

Beamsearch enhances the search algorithm by expanding a group of additional nodes (referred to as the *beam*) alongside the closest node, with the goal of improving SSD utilization. The beamwidth B refers to the number of nodes in a beam. However, the extra nodes selected for expansion are not always critical to the search. For example, the neighbors of the closest node may have greater relevance to the query than the nodes chosen for beam expansion. In such scenarios, sequentially expanding one node at a time may be more efficient, avoiding unnecessary use of SSD resources on irrelevant nodes.

Beamsearch introduces an inherent delay since the expansion of the next closest node occurs only after the current beam request is fully processed. If the beam includes irrelevant nodes, this delay is compounded, further slowing search traversal. Additionally, SSDs are susceptible to read retries caused by read disturbances, leading to unpredictable latency. These variations exacerbate inefficiencies in node processing, resulting in increased overall search latency.

B. GraphAccel-Search High-level Overview

GraphAccel's search algorithm leverages the abundant parallelism in SSDs by employing speculative search executions. Similar to the Beamsearch mechanism used in DiskANN, GraphAccel-Search issues additional node expansion requests. However, these additional requests are handled as *speculative* rather than essential. The key distinction of GraphAccel-Search is that the SSD controller categorizes requests into two types:

1) **Normal requests**, associated with the closest unexpanded node, are processed without dropping.
2) **Droppable requests**, associated with speculative node expansions, are processed opportunistically.

If the requested SSD resources—-such as the corresponding channels or flash chips-—are occupied when a droppable request is issued, the SSD controller cancels the request. This ensures that critical resources are preserved for processing normal requests.

Since speculative requests may be partially dropped, nodes expanded during speculative executions can remain in an incomplete state. To maintain accuracy, GraphAccel-Search includes a *re-examination phase*, analogous to the verification or correction step in speculative execution pipelines in CPUs. This phase enhances the quality of results while maximizing parallelism in earlier stages.

The re-examination phase is performed in two scenarios:

1) **Natural re-examination** occurs when a speculatively expanded node becomes the closest unexpanded node during subsequent steps of the search.
2) **Batch re-examination** occurs when all nodes in the search list are either fully expanded or speculatively expanded. At this stage, the algorithm revisits the remaining speculatively expanded nodes to resolve any incomplete states. During this process, all requests are treated as normal to ensure accuracy.

Although the number of nodes requiring re-examination can be large, especially with a larger Beamwidth, the impact on performance is mitigated. This is because most neighbor nodes are already accessed during speculative executions, requiring only a few additional flash accesses during the re-examination step.

C. GraphAccel-Search Details

Figure 4 illustrates how GraphAccel-Search handles node expansions during different phases of query processing.

Figure 4a shows the status after the initial entry node has expanded. The initial entry node is marked as expanded, and its neighbors are added to the search list. Among these neighbors, the one closest to the query, as determined by the PQ distance, is selected for non-droppable expansion. Meanwhile, the next B-1 closest neighbors are selected for speculative expansion, which can be dropped if necessary. Droppable requests, indicated by dotted lines in the figure, may be canceled by the SSD controller during flash access processing. Once the neighbor PQ vectors are fetched, these neighbors are added to the search list with their PQ distances. Upon completing the batch of droppable and non-droppable requests, the closest node is marked as fully expanded (red), while the B-1 speculatively expanded nodes are marked for re-examination later (blue).

Figure 4b shows the middle phase of the search process. At this stage, the search list contains fully expanded nodes, speculatively expanded nodes, and unexpanded nodes. For the next requests, the closest unexpanded or speculatively expanded node is selected for non-droppable expansion. If a speculatively expanded node is chosen, this effectively re-examines the earlier speculation. Alongside the non-droppable request, B-1 unexpanded nodes are selected for speculative expansion. Importantly, nodes that have already undergone speculative expansion are not selected for speculation again.

Figure 4c depicts the phase when all nodes are either fully expanded or speculatively expanded. In this phase, B^* nodes are expanded in a batch, all treated as non-droppable requests. This batch expansion focuses on re-examining speculative nodes to finalize their evaluation. Since most pages have already been accessed by this stage, fewer flash page requests are generated, allowing for a

larger batch size to maximize SSD parallelism. In our experiments, we empirically set B^* to 100 to fully exploit available parallelism.

The search concludes when all nodes in the search list are fully expanded. If batch re-examination reintroduces unexpanded nodes to the search list, the algorithm repeats the speculative expansion and batch re-examination phases until full expansion is achieved.

Note that not all speculatively expanded nodes proceed to the re-examination phase. If a speculatively expanded node is removed from the search list of size L by closer nodes to the query, it is not re-examined. This selective re-examination enhances the efficiency of our opportunistic evaluation strategy for speculative expansion, as opposed to uniformly exploring all B nodes in the beam.

D. GraphAccel-Search SSD Structure

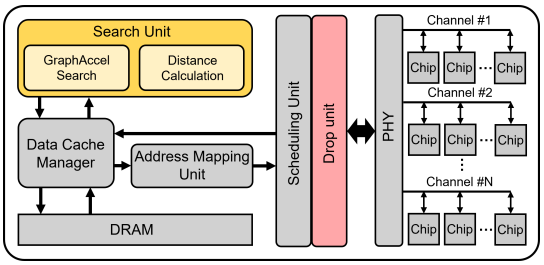


Fig. 5: High-level view of the components of GraphAccel.

Figure 5 illustrates the high-level interactions between the components of GraphAccel. To enhance vector search performance using ISP capabilities, two new components are introduced: the *Search Unit*, which executes the search algorithm and performs distance calculations, and the *Drop Unit*, which manages the cancellation of speculative droppable requests at the time of request issuance.

A dedicated data path between the Search Unit and the *Data Cache Manager* (DCM) enables efficient communication. If the requested data is located in DRAM, the DCM retrieves it and forwards it directly to the Search Unit for computation, avoiding unnecessary fetch operations from the flash chip.

When requests are submitted to the Scheduling Unit for chip roundtrips, the Drop Unit evaluates each request to determine whether it is droppable. For droppable requests, the Drop Unit verifies the status of the target chip and channel, processing the request only if both are idle, thereby optimizing resource utilization.

V. Evaluation

A. Experimental Setup

Implementation. We implement GraphAccel within MQSim [22], a widely used simulator for modern NVMe SSDs. We model the specification of SSD as below Table II. The SSD internal DRAM is set to 1 GB, with a storage capacity of 1 TB.

The newly integrated Search Unit is designed to operate using the same microcontroller as the flash translation layer of the SSD controller. We estimate its compute latency by profiling execution times on an ARM Cortex-A9 core processor running at a clock frequency of 200 MHz.

To model SSD read retries, we utilize measurement data from the literature [17], focusing on three retention age scenarios: 0 months, 3 months, and 6 months.

Datasets. We selected three datasets, Turing100M, SIFT1B, and DEEP1B, for evaluating large-scale vector searches, each containing over 100 million nodes. The details of these datasets are summarized in Table III. The search performance is evaluated by measuring the recall rate (R@k) against the total latency for processing queries from each dataset. This performance is compared with existing vector search algorithms, including DiskANN and DiskANN++, all evaluated under the same ISP environment as GraphAccel.

TABLE II: System Configurations

System	Config	Value
Search Unit	Clock Frequency	200MHz
Memory	Capacity	1GB
	Bandwidth	2GB/s
Storage	Topology	8 Channels, 4 Chips / Channel
	Capacity	1TB
	Read Latency	50us per single page

TABLE III: Dataset Information

Dataset	# Node	# Dimension	Type	Size (GB)
Turing100M [11]	100,000,000	100	float32	38
SIFT1B [12]	1,000,000,000	128	uint8	119
DEEP1B [13]	1,000,000,000	96	float32	358

B. Experimental Results

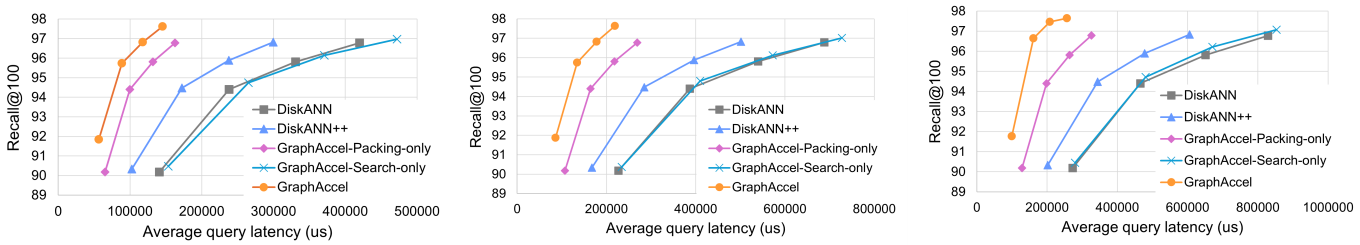
Resistance to Variable Read Latencies. Figure 6 compares the performance of GraphAccel, DiskANN, and DiskANN++ using the SIFT1B dataset, with the beamwidth of 16. We varied the search list length, L, from 500 to 2,000 in increments of 500 to analyze the trade-offs between search latency and recall.

GraphAccel demonstrates superior performance, achieving higher recall values with shorter overall latency. For example, at a recall level of approximately 97%, GraphAccel achieves a search latency reduction of 2.54×, 2.28×, and 2.36× compared to DiskANN++ for retention ages of 0, 3, and 6 months, respectively. With more read retries, GraphAccel excels by efficiently scheduling requests to maximize hardware utilization. When certain chips experience delayed latency, GraphAccel dynamically mitigates the impact of these delays through effective resource management.

In addition to reducing latency, GraphAccel-Search also improves recall. This improvement can be attributed to the increased number of node expansions in GraphAccel for the same search list length. By assigning a lower priority to additional node expansions, GraphAccel accelerates transitions to subsequent iterations of node expansion. As a result, GraphAccel-Search more frequently expands the most relevant portions of the search graph using the same resources, rather than waiting to fill the search list with less important nodes.

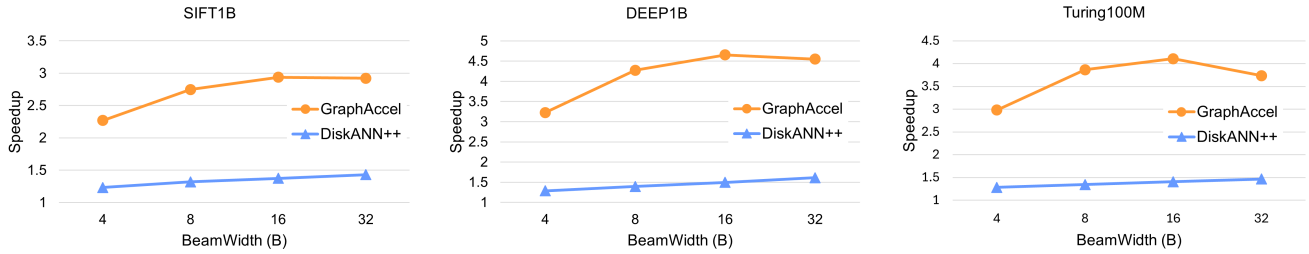
Effect of Beamwidth. As shown in Fig 7, as the beamwidth increases, more requests are processed simultaneously, allowing GraphAccel to significantly outperform DiskANN and DiskANN++. The slight decline in performance speedup observed as the beam size increases from 16 to 32 is attributed to the limitations of hardware resources. Our simulation setup includes 8 channels, each configured with 4 chips, resulting in a total of 32 chips available to handle concurrent requests. When the beam size remains within the hardware's parallelism limitations, increasing the beam size enables our GraphAccel to achieve significant performance improvements across all datasets. This improvement is attributed to its ability to effectively prioritize primary requests while discarding additional speculative requests that might otherwise increase the overall latency.

Page Accesses. Figure 8 depicts the average number of accessed pages per query normalized to DiskANN across different datasets, including SIFT1B, DEEP1B, and Turing100M. While DiskANN++



(a) (b) (c)

Fig. 6: Latency (microseconds) vs. Recall@100 for DiskANN, DiskANN++, and GraphAccel: (a) Retention age of 0 months, (b) Retention age of 3 months, (c) Retention age of 6 months.



(a) (b) (c)

Fig. 7: Speedups achieved with varying Beamwidth (B) compared to DiskANN across datasets: (a) SIFT1B, (b) DEEP1B, (c) Turing100M

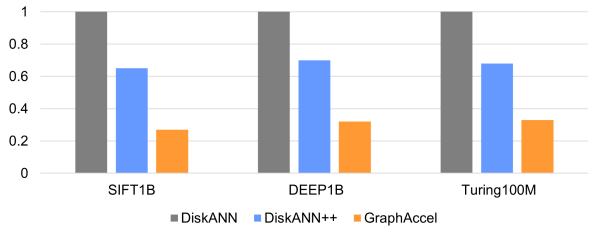


Fig. 8: Normalized Average Number of Flash Page Accesses per Query (Relative to DiskANN)

outperforms DiskANN due to its implementation of Starpacking, it still requires accessing a substantial number of pages. In contrast, GraphAccel demonstrates approximately 69.4% fewer accessed pages per query across all three datasets compared to DiskANN, up to 58.4% to DiskANN++. This result highlights the versatility of our GraphAccel-packing method, demonstrating its effectiveness across datasets with diverse characteristics.

VI. Related Work

VStore is an in-storage, graph-based vector search accelerator that leverages hardware optimizations, query orchestration, and result reuse strategies [23]. However, its performance heavily depends on temporal locality between multiple queries. In scenarios where queries are less predictable or exhibit poor locality, its effectiveness may be significantly reduced.

Pyramid is an in-storage ANN search acceleration platform with a hierarchical memory architecture [24], offering more than an order-of-magnitude speedup compared to CPU- or GPU-based systems. However, its design requires storing the compacted graph in memory, limiting the graph size to the capacity of the main memory. In contrast, GraphAccel stores all graph data on storage, eliminating graph size constraints except those imposed by storage capacity.

Kim et al. and Tian et al. have developed ANN search systems utilizing computational storage devices, specifically SmartSSDs [25], [26]. These systems achieve scalability by distributing the search workload across multiple SmartSSD units. However, due to their reliance on commercial platforms with fixed SSD controllers, their scheduling lacks the aggressive optimizations in GraphAccel's speculative search mechanism.

VII. Conclusion

In this paper, we introduced GraphAccel, an in-storage accelerator tailored for efficient graph-based vector similarity search. To address key challenges associated with large-scale searches on SSDs, GraphAccel incorporates an optimized page packing mechanism to reduce SSD page accesses and a speculative search strategy to fully exploit SSD parallelism. These techniques collectively minimize search latency while maintaining high recall accuracy, even under conditions of variable SSD read latency.

Our evaluation on large-scale datasets, including Turing100M, SIFT1B, and DEEP1B, demonstrates the significant performance gains of GraphAccel over state-of-the-art methods like DiskANN and DiskANN++. Moreover, GraphAccel's dynamic resource utilization and speculative execution strategies ensure resilience to retention-age-induced read disturbances, further underscoring its suitability for large-scale and latency-sensitive applications.

Acknowledgments

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grants funded by the Korea government (MSIT) (No. RS-2022-II221199, No. RS-2024-00398745, and No. RS-2024-00454666) and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2023-00251287).

REFERENCES

[1] A. Sharif Razavian *et al.*, "CNN features off-the-shelf: An astounding baseline for recognition," in *CVPR Workshops*, 2014.

[2] D. Cer, "Universal sentence encoder," *arXiv preprint arXiv:1803.11175*, 2018.

[3] T. Bertin-Mahieux *et al.*, "The million song dataset," 2011.

[4] J. Jo *et al.*, "PANENE: A progressive algorithm for indexing and querying approximate k-nearest neighbors," *IEEE TVCG*, vol. 26, no. 2, pp. 1347–1360, 2020.

[5] W. Li *et al.*, "Approximate nearest neighbor search on high dimensional data - experiments, analyses, and improvement," *IEEE TKDE*, vol. 32, no. 8, pp. 1475–1488, 2020.

[6] Q. Wang *et al.*, "Fast-adapting and privacy-preserving federated recommender system," *The VLDB Journal*, 2022.

[7] J. Wang *et al.*, "Scalable k-NN graph construction for visual descriptors," in *CVPR*, 2012.

[8] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *NeurIPS*, 2020.

[9] S. J. Subramanya *et al.*, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *NeurIPS*, 2019.

[10] Y. A. Malkov *et al.*, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE TPAMI*, vol. 42, no. 4, p. 824–836, Apr. 2020.

[11] H. Zhang *et al.*, "Generic intent representation in web search," in *SIGIR*, 2019.

[12] M. D. Hervé Jégou, Romain Tavenard *et al.*, "Searching in one billion vectors: re-rank with source coding," in *ICASSP*, 2011.

[13] A. B. Yandex *et al.*, "Efficient indexing of billion-scale datasets of deep descriptors," in *CVPR*, 2016.

[14] J. Ni *et al.*, "DiskANN++: Efficient page-based search over isomorphic mapped graph index using query-sensitivity entry vertex," 2023, arXiv:2310.00402.

[15] Y. Cai *et al.*, "Read disturb errors in mlc nand flash memory: Characterization, mitigation, and recovery," in *DSN*, 2015.

[16] Y. Luo *et al.*, "Improving 3d nand flash memory lifetime by tolerating early retention loss and process variation," *Proc. ACM Meas. Anal. Comput. Syst.*, 2018.

[17] J. Park *et al.*, "Reducing solid-state drive read latency by optimizing read-retry," in *ASPLOS*, 2021.

[18] P. Kumari *et al.*, "Radiation-induced error mitigation by read-retry technique for mlc 3-d nand flash memory," *IEEE TNS*, 2021.

[19] M. Ye *et al.*, "Achieving near-zero read retry for 3d nand flash memory," in *ASPLOS*, 2024.

[20] C. S. Hervé Jégou, Matthijs Douze, "Product Quantization for Nearest Neighbor Search," in *IEEE TPAMI*, 2011.

[21] G. M. c *et al.*, "Partitioning trillion-edge graphs in minutes," in *IPDPS*, 2017.

[22] A. Tavakkol *et al.*, "MQSim: A Framework for Enabling Realistic Studies of Modern Multi-Queue SSD Devices," in *FAST*, 2018.

[23] S. Liang *et al.*, "VStore: in-storage graph based vector search accelerator," in *DAC*, 2022.

[24] Z. Zhu *et al.*, "Processing-in-hierarchical-memory architecture for billion-scale approximate nearest neighbor search," in *DAC*, 2023.

[25] J.-H. Kim *et al.*, "Accelerating large-scale graph-based nearest neighbor search on a computational storage platform," *IEEE TC*, vol. 72, no. 1, pp. 278–290, 2023.

[26] B. Tian *et al.*, "Scalable billion-point approximate nearest neighbor search using SmartSSDs," in *USENIX ATC 24*, 2024.